
Data Preprocessing

Data Preprocessing

- Why preprocess the data?
- Data cleaning
- Data integration and transformation
- Data reduction
- Discretization and concept hierarchy generation
- Summary

Why Data Preprocessing?

- Data preprocessing is a crucial step in the data analysis and machine learning pipeline because raw data often contains imperfections or complexities that make it unsuitable for direct use.
- Data in the real world is dirty
 - **Incomplete data**: lacking attribute values, lacking certain attributes of interest, or containing only aggregate data
 - e.g., occupation=" "
 - **Noisy data**: containing errors or outliers
 - e.g., Salary=" -10"
 - **Inconsistent data**: containing discrepancies in codes or names
 - e.g., Age=" 42" Birthday=" 03/07/1997"
 - e.g., Was rating " 1,2,3" , now rating " A, B, C"
 - e.g., discrepancy between duplicate records

Why Data Preprocessing?

- Here's why data preprocessing is required:

1. Improving Data Quality

- **Handling Missing Values:** Real-world datasets often have missing or incomplete data. Preprocessing helps fill in missing values (e.g., using mean, median, or predictive methods) or remove incomplete rows/columns.
- **Removing Noise:** Outliers, and irrelevant data can skew the analysis. Preprocessing identifies and removes such noise to ensure data quality.

Why Data Preprocessing?

2. Ensuring Consistency

- **Standardization/Normalization:** Features in datasets may have different scales or units (e.g., age in years and income in dollars). Standardization or normalization makes these features comparable by scaling them to a consistent range or distribution.
- **Data Transformation:** Converting data into a consistent format (e.g., dates into a unified format or text data into numerical representations) ensures compatibility and improves interpretability.

Why Data Preprocessing?

3. Enhancing Model Performance

- **Feature Selection and Extraction:** Identifying and focusing on the most relevant features reduces dimensionality and improves model accuracy while decreasing computational complexity.
- **Removing Correlations:** Highly correlated features can cause redundancy and degrade model performance. Preprocessing helps identify and address these issues.

Why Data Preprocessing?

4. Facilitating Data Understanding

- **Data Cleaning:** Cleaning data by addressing inconsistencies or formatting errors helps analysts and stakeholders understand the dataset better.
- **Visualization:** Preprocessed data is often easier to visualize, enabling clearer insights and better communication of findings.

5. Reducing Computational Overheads

- **Reducing Dimensionality:** By selecting only the most critical features, preprocessing decreases the dataset's size, leading to faster training times and reduced resource consumption.

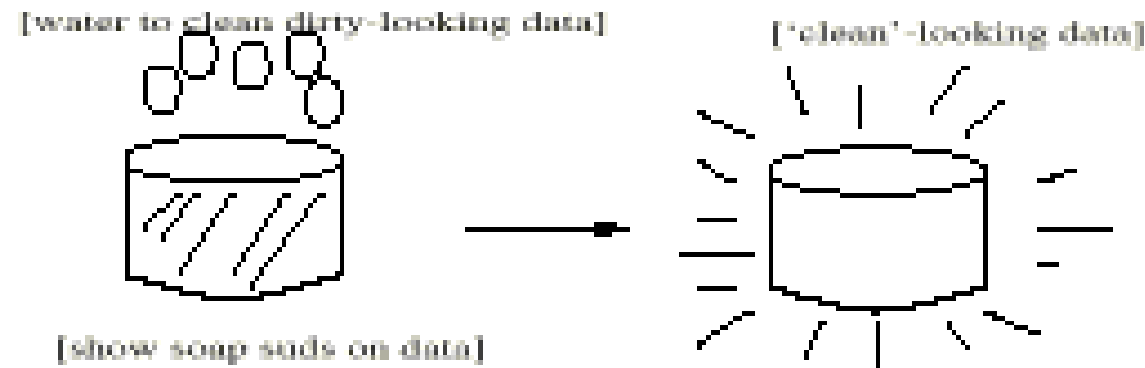
Why Data Preprocessing?

6. Improving Model Generalization

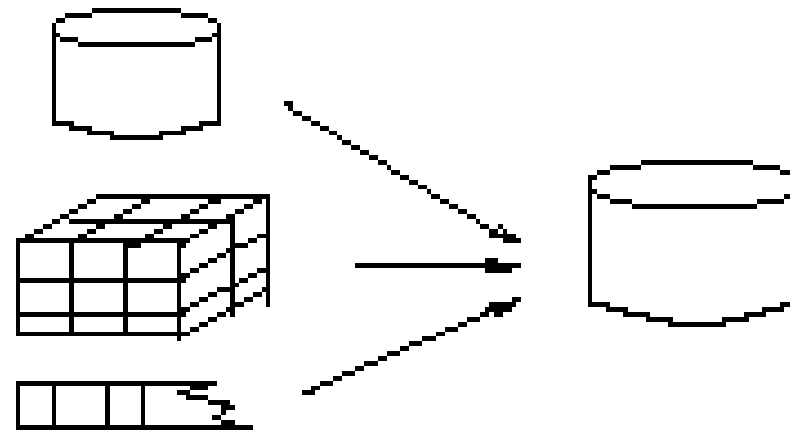
- **Reducing Overfitting:** Techniques like feature scaling or data augmentation prevent models from learning noise in the data, enhancing their ability to generalize to unseen data.
- **Balancing Classes:** Addressing class imbalances (e.g., through resampling methods like oversampling or undersampling) ensures the model learns equally from all categories.
- In summary, data preprocessing transforms raw data into a clean, consistent, and usable form that improves the reliability and performance of downstream analysis or machine learning models.

Forms of Data Preprocessing

Data Cleaning



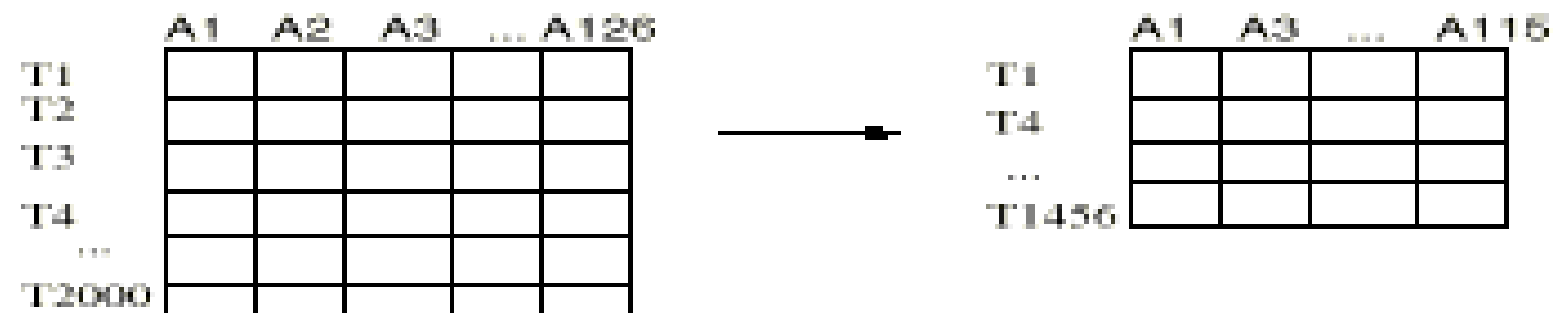
Data Integration



Data Transformation

-2, 32, 100, 59, 48 → -0.02, 0.32, 1.00, 0.59, 0.48

Data Reduction



Data Cleaning

- Importance
 - Data cleaning is one of the biggest problems in data warehousing – Ralph Kimball
 - Data cleaning is the number one problem in data warehousing – DCI survey
- Data cleaning tasks
 - Fill in missing values
 - Identify outliers and smooth out noisy data
 - Correct inconsistent data
 - Resolve redundancy caused by data integration

Missing (incomplete) Data

- Data is not always available
 - E.g., many tuples have no recorded value for several attributes, such as customer income in sales data
- Missing data may be due to
 - equipment malfunction
 - inconsistent with other recorded data and thus deleted
 - data not entered due to misunderstanding
 - certain data may not be considered important at the time of entry
 - not register history or changes of the data
- Missing data may need to be inferred.

How to Handle Missing Data?

- Missing value imputation is the process of replacing missing data with substituted values to ensure the dataset remains useful for analysis or modeling.
- Different techniques for imputing missing values can be categorized into **basic methods**, **statistical methods**, and **machine learning-based methods**:

1. Handle Missing Data: Basic Techniques

1. Basic Techniques

(i) Mean/Median/Mode Imputation

- Replace missing values with:
 - **Mean** (average) for numerical data.
 - **Median** (middle value) if the data is skewed.
 - **Mode** (most frequent value) for categorical data.
- Replace missing values with for all samples belonging to the same class: smarter
 - **Mean** (average) for numerical data.
 - **Median** (middle value) if the data is skewed.
 - **Mode** (most frequent value) for categorical data.
- **Pros:** Simple and quick.
- **Cons:** Reduces variance and may distort relationships.

1. Handle Missing Data: **Basic Techniques**

(ii) Arbitrary Value Imputation

- Replace missing values with an arbitrary value, such as 0 or -999.
- **Pros:** Easy to implement; creates a distinct value for missing data.
- **Cons:** Can introduce bias if the chosen value affects the analysis.

(iii) Constant Value Imputation

- Replace all missing values with a constant defined by domain knowledge.
- Example: Replacing missing ages in a dataset with "Unknown", a new class?!

2. Handle Missing Data: **Statistical Methods**

(i) K-Nearest Neighbors (KNN) Imputation

- Replace missing values with values from the nearest neighbors in feature space.
- **Pros:** Considers relationships between variables.
- **Cons:** Computationally expensive for large datasets.

(ii) Regression Imputation

- Predict missing values using regression models based on other variables.
- Example: Use a linear regression model to estimate missing values of Feature A using Feature B.
- **Pros:** Considers dependencies among variables.
- **Cons:** Risk of overfitting; sensitive to model choice.

3. Machine Learning-Based Methods

(i) Decision Tree/Random Forest Imputation

- Train a decision tree or random forest model to predict missing values using other features. (Leave nodes are the values of the attribute with missing value)
- **Pros:** Handles complex relationships and interactions.
- **Cons:** Computationally intensive.

(ii) Deep Learning-Based Imputation

- Use neural networks like autoencoders to learn patterns in data and impute missing values.
- **Pros:** Captures non-linear relationships.
- **Cons:** Requires large datasets and significant computational resources.

Noisy Data

- Noise: random error or variance in a measured variable
- Incorrect attribute values may due to
 - faulty data collection instruments
 - data entry problems
 - data transmission problems
 - technology limitation
 - inconsistency in naming convention

How to Handle Noisy Data?

- Binning

- first sort data and partition into (equal-frequency) bins
- then one can smooth by bin means, smooth by bin median, smooth by bin boundaries, etc.

- Regression

- smooth by fitting the data into regression functions

- Clustering

- detect and remove outliers

- Combined computer and human inspection

- detect suspicious values and check by human (e.g., deal with possible outliers)

Simple Discretization Methods: Binning

- **Equal-width** (distance) partitioning
 - Divides the range into N intervals of equal size: uniform grid
 - if A and B are the lowest and highest values of the attribute, the width of intervals will be: $W = (B - A)/N$.
- **Advantages:**
 - (i) Easy to understand and visualize, especially when working with evenly distributed data.
 - (ii) Computationally efficient since it only involves defining a fixed width and counting data points within each range.
 - (iii) Suitable for evenly distributed datasets.

Equal-width (distance) partitioning

■ Disadvantages:

- (i) Outliers may dominate presentation
 - (ii) Ineffective for skewed or non-uniform data, as some bins may contain no data, while others may contain too many points.
 - (iii) May fail to capture meaningful patterns in highly variable datasets.
- **Example:** For the dataset [1, 2, 2, 3, 6, 8, 12, 15, 19], dividing into three equi-width bins with range width 6 would result in bins like [1, 7), [7, 13), and [13, 19].

Thus the partitions are: [1, 2, 2, 3, 6], [8, 12] and [15, 19]

Simple Discretization Methods: Binning

- **Equal-depth** (frequency) partitioning
 - Sort the data in ascending order.
 - Choose the number of bins (k).
 - Divide the data into k bins, each containing approximately n/k data points, where n is the total number of data points.
- **Advantages:**
 - Ensures a balanced distribution of data points across bins.
 - Handles skewed data well.
- **Disadvantages:**
 - Bin widths can vary, making intervals harder to interpret.
 - Can be computationally more intensive for large datasets due to sorting.

Equal-depth (frequency) partitioning

- **Example:**
- Sorted Data: [10, 15, 20, 25, 30, 35, 40]
- Number of bins (k): 3
- Number of data points per bin: $7/3 \approx 2$
- Bins:
 - Bin 1: [10, 15]
 - Bin 2: [20, 25]
 - Bin 3: [30, 35, 40]

Key Differences between Equidepth and Equiwidth partitioning

Aspect	Equidepth Partitioning	Equiwidth Partitioning
Definition	Equal number of data points in each bin.	Equal range (width) of bins.
Range of Bins	Variable, depends on data distribution.	Fixed, defined by a constant width.
Data Distribution	Effective for skewed/non-uniform data.	Effective for uniform data.
Computational Complexity	Higher, involves sorting or density calculations.	Lower, simple and direct implementation.
Interpretability	May be less intuitive due to varying ranges.	More intuitive, especially in histograms.

Binning Methods (equi-width) for Data Smoothing

- Data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34
- Range = $34 - 4 = 30$.
- Let 3 bins are used.
- Bin width = $30/3=10$
- Bin 1 = [4, 14), Bin 2 = [14, 24), Bin 3 = [24, 34]

* Partition into equi-width bins:

- Bin 1: 4, 8, 9
- Bin 2: 15, 21, 21
- Bin 3: 24, 25, 26, 28, 29, 34

* Smoothing by bin means:

- Bin 1: 7, 7, 7
- Bin 2: 19, 19, 19
- Bin 3: 28, 28, 28, 28, 28, 28

*

Binning Methods (equi-width) for Data Smoothing

- Partition into equi-width bins:
 - Bin 1: 4, 8, 9
 - Bin 2: 15, 21, 21
 - Bin 3: 24, 25, 26, 28, 29, 34

- Smoothing by bin boundaries:
 - Bin 1: 4, 9, 9
 - Bin 2: 15, 21, 21
 - Bin 3: 24, 24, 24, 24, 24/34, 34

Binning Methods (equi-depth) for Data Smoothing

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34
- * Partition into equal-frequency (equi-depth) bins:
 - Bin 1: 4, 8, 9, 15
 - Bin 2: 21, 21, 24, 25
 - Bin 3: 26, 28, 29, 34
- * Smoothing by bin means:
 - Bin 1: 9, 9, 9, 9
 - Bin 2: 23, 23, 23, 23
 - Bin 3: 29, 29, 29, 29
- * Smoothing by bin boundaries:
 - Bin 1: 4, 4, 4, 15
 - Bin 2: 21, 21, 25, 25
 - Bin 3: 26, 26, 26, 34

Error in smoothing

- Original data: 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34
- Total variation = $4+1+6+6+0+3+1+1+2+1+5=30$
- Equi-width: 7, 7, 7, 19, 19, 19, 28, 28, 28, 28, 28, 28
- Total variation = $0+0+12+0+0+9+0+0+0+0+0+0=21$
- Equi-depth: 9, 9, 9, 9, 23, 23, 23, 23, 29, 29, 29, 29
- Total variation = $0+0+0+14+0+0+0+6+0+0+0+0=20$
- Lower total variation indicates smoother data.

Binning by Mean for skewed Data Smoothing

- Data for price (in dollars): 4, 8, 9, 15, 21, 21, 54, 56, 86, 88, 88, 150
- * Partition into equi-width bins: (Bin width~ 49)
 - Bin 1: 4, 8, 9, 15, 21, 21
 - Bin 2: 54, 56, 86, 88, 88
 - Bin 3: 150
- * Smoothing by bin means:
 - Bin 1: 13, 13, 13, 13, 13, 13
 - Bin 2: 93, 93, 93, 93, 93
 - Bin 3: 150
- * Original data: 4, 8, 9, 15, 21, 21, 54, 56, 86, 88, 88, 150
- * Total variation=146
- * Equi-width : 13, 13, 13, 13, 13, 13, 93, 93, 93, 93, 93, 150
- *Total variation= 0+0+0+0+0+80+0+0+0+0+57=137

Binning by Mean for skewed Data Smoothing

- Sorted data for price (in dollars): 4, 8, 9, 15, 21, 21, 54, 56, 86, 88, 88, 150
- * Partition into equi-depth bins:
 - Bin 1: 4, 8, 9, 15
 - Bin 2: 21, 21, 54, 56
 - Bin 3: 86, 88, 88, 150
- * Smoothing by bin means:
 - Bin 1: 9, 9, 9, 9
 - Bin 2: 38, 38, 38, 38
 - Bin 3: 103, 103, 103, 103
- * Original data: 4, 8, 9, 15, 21, 21, 54, 56, 86, 88, 88, 150
- * Total variation= $4+1+6+6+0+33+2+30+2+0+62=146$
- * Equi-depth : 9, 9, 9, 9, 38, 38, 38, 38, 103, 103, 103, 103
- * Total variation= $0+0+0+29+0+0+0+65+0+0+0=94$

Regression for Data Smoothing

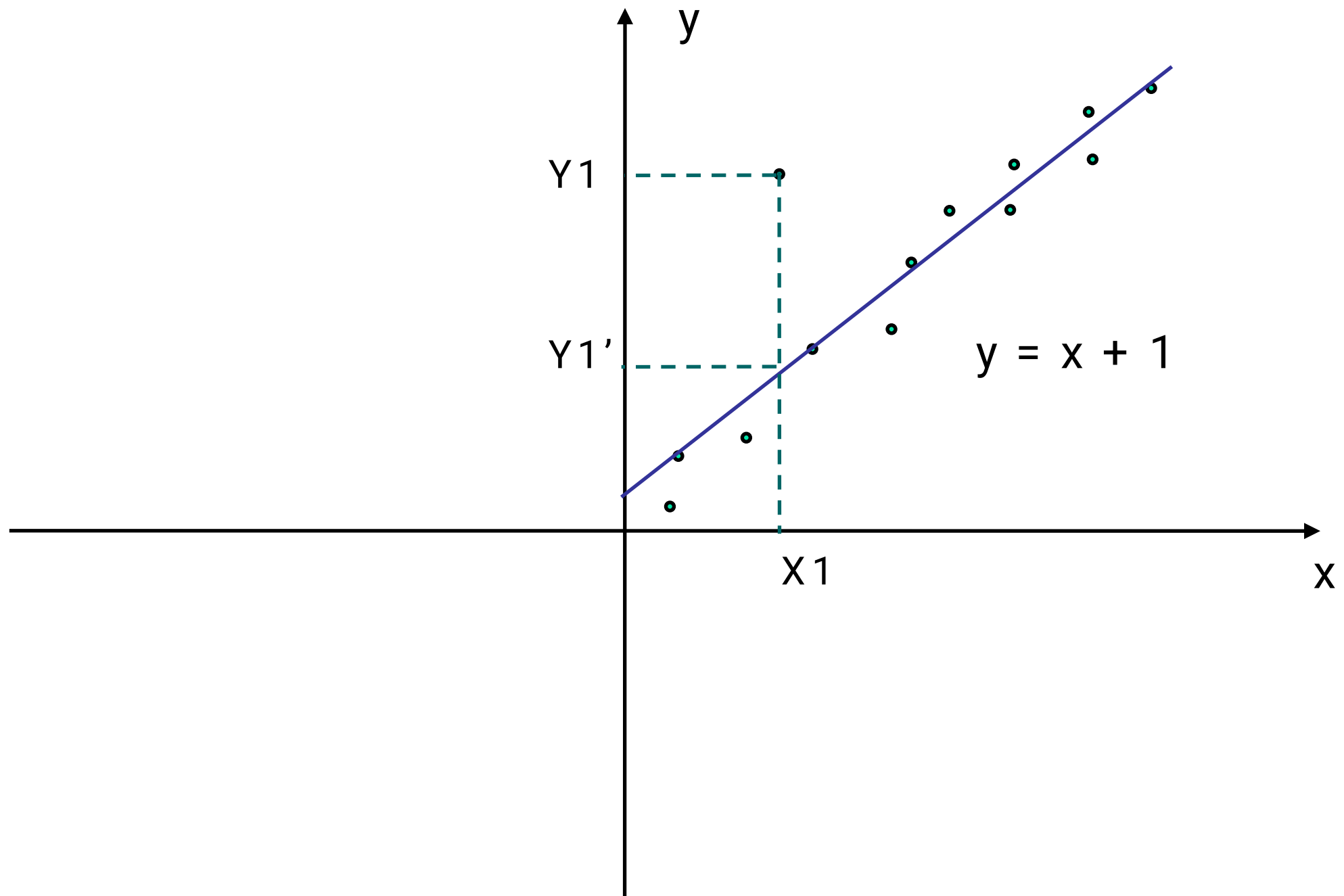
1. Purpose of Data Smoothing:

- Data smoothing involves removing random noise or outliers in a dataset to reveal the overall trend or structure. It helps in:
 - Enhancing the interpretability of data.
 - Preparing data for modeling or analysis.
 - Reducing overfitting in predictive models.

2. Role of Regression in Smoothing:

- Regression models fit a mathematical function to the data, which serves as a smoothed representation.
- The choice of regression type depends on the nature of the data and the desired smoothness.

Regression for Data Smoothing



3. Steps in Using Regression for Smoothing

- **Understand the Data:**

- Analyze the data to decide the type of smoothing needed (linear, non-linear, localized).

- **Choose the Regression Model:**

- Select the appropriate model (e.g., linear, polynomial, LOESS) based on data patterns.

- **Fit the Model:**

- Train the regression model using the dataset.

- **Generate Smoothed Values:**

- Use the fitted model to predict or interpolate smoothed values for the data points.

- **Evaluate Smoothness:**

- Assess whether the smoothed data preserves the trend while reducing noise.

Regression for Data Smoothing

4. Benefits of Using Regression for Smoothing

- **Flexibility:** A wide range of regression models can suit various data types.
- **Interpretability:** Provides a clear mathematical representation of the smoothed data.
- **Noise Reduction:** Removes random fluctuations while retaining essential trends.
- By selecting the right regression technique, we can achieve an effective balance between noise reduction and trend preservation, enhancing the data's usability for further analysis.

Regression for Data Smoothing

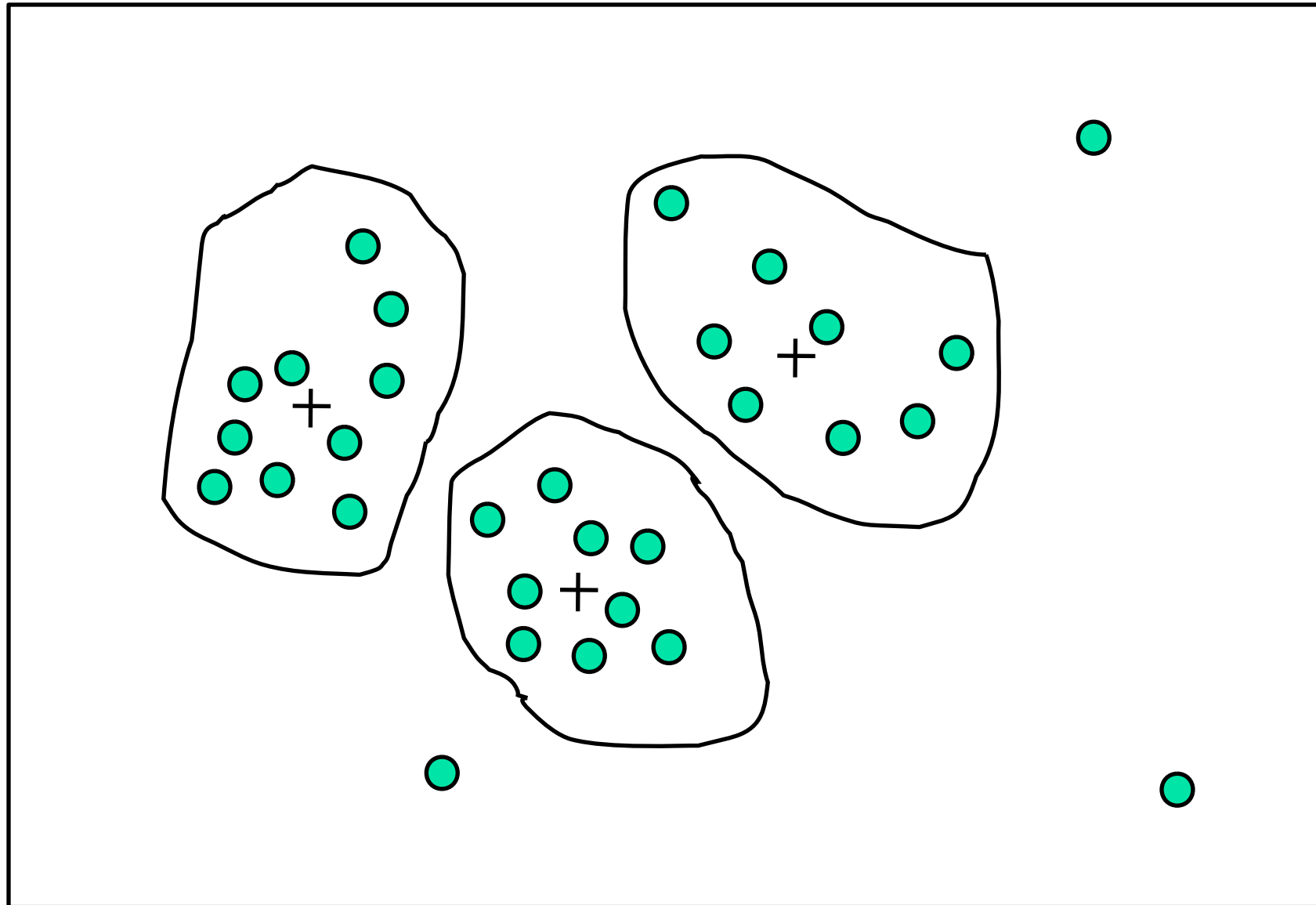
5. Applications

- **Time Series Analysis:** Smoothing economic or financial data for forecasting.
- **Signal Processing:** Reducing noise in audio or image signals.
- **Environmental Studies:** Analyzing temperature or pollution trends.
- **Healthcare:** Smoothing patient health metrics over time.

Clustering for Data Smoothing

- **Clustering for data smoothing** involves grouping data points based on similarity and then using the cluster-level characteristics (e.g., centroids or means) to replace or summarize individual data points.
- This approach reduces noise and variability while retaining the overall structure or pattern in the data.

Clustering for Data Smoothing



How Clustering Enables Data Smoothing

- Clustering identifies groups (clusters) of similar data points in the dataset. Once clusters are formed:
 - (i) The **centroids** (mean or representative points of clusters) can be used to approximate the values of individual data points within a cluster.
 - (ii) This reduces fluctuations and highlights the overall trend by consolidating data within groups.

Example of Clustering for Smoothing

- **Input Data:**
- Data Points: [12, 15, 20, 22, 50, 52, 55]
- **Using K-Means with $k=2$:**
- Cluster 1: [12, 15, 20, 22], Centroid = 17.25
- Cluster 2: [50, 52, 55], Centroid = 52.33
- **Smoothed Data:**
- [17.25, 17.25, 17.25, 17.25, 52.33, 52.33, 52.33]
- This results in reduced variability and a clearer overall trend.

Steps in Clustering-Based Smoothing

- **Choose a Clustering Algorithm:** Use algorithms such as **K-means**, **hierarchical clustering**, or **DBSCAN** depending on the data's nature.
- **Cluster the Data:** Partition the dataset into clusters based on a similarity measure (e.g., Euclidean distance).
- **Replace Data with Cluster Representatives:** Replace each data point in a cluster with the cluster centroid (or another representative value like median or mode).
- **Evaluate the Smoothed Data:** Assess whether the smoothed data reduces noise while preserving trends.

Advantages of Clustering for Data Smoothing

- **Noise Reduction:**

- Outliers and random fluctuations are minimized by consolidating data within clusters.

- **Pattern Recognition:**

- Clustering naturally highlights underlying patterns or groupings in the data.

- **Adaptability:**

- Can handle non-linear trends and relationships in data, especially with density-based methods.

Disadvantages of Clustering for Smoothing

- **Loss of Granularity:**

- Individual data points are replaced by cluster-level summaries, which may oversimplify the data.

- **Cluster Choice Sensitivity:**

- Results depend on the choice of clustering parameters (e.g., number of clusters, distance metric).

- **Computational Complexity:**

- Clustering large datasets can be computationally intensive.

Applications of Clustering for Smoothing

- **Time Series Data:** Smooth noisy trends in sales or weather data by clustering time periods with similar behaviors.
- **Spatial Data:** Aggregate geographical data into regions and use cluster summaries for visualization.
- **Image Processing:** Cluster pixel intensities to smooth images and reduce noise.
- **Healthcare:** Smooth patient metrics (e.g., blood pressure readings) by grouping similar readings.
- **Conclusion:** Clustering-based smoothing is particularly effective for datasets with inherent groupings or repeated patterns. While it simplifies noisy data, careful selection of clustering parameters and algorithms ensures that trends are preserved without overgeneralization.

Inconsistent Data

- Handling inconsistent data is a crucial step in data preprocessing to ensure that the dataset is reliable and suitable for analysis or modeling.
- Inconsistent data refers to discrepancies, irregularities, or errors in the dataset, such as mismatched formats, duplicate records, conflicting values, or typographical errors.

How to Identify Inconsistencies?

Before handling inconsistent data, identify the types and locations of inconsistencies:

- **Data Type Inconsistencies:** Values that don't match the expected type (e.g., text in a numeric column).
- **Format Issues:** Variations in formats (e.g., dates written as "01/01/2025" and "2025-01-01").
- **Duplicate Records:** Repeated entries in the dataset.
- **Conflicting Values:** Different values for the same attribute in similar records (e.g., "NY" vs. "New York").
- **Typographical Errors:** Misspelled words or numeric errors (e.g., "Applee" instead of "Apple").
- **Case Sensitivity Issues:** Variations due to capitalization (e.g., "John Doe" vs. "john doe").
- **Missing Data:** Empty or incomplete entries that lead to inconsistencies.

Techniques to Handle Inconsistent Data

(i) Standardize Formats

- Ensure consistency in formats across the dataset:
 - **Dates:** Convert all dates to a standard format, e.g., YYYY-MM-DD.
 - **Units:** Standardize units of measurement (e.g., "5 kg" to "5000 g").
 - **Text Case:** Normalize text to a consistent case (e.g., all lowercase or title case).

Techniques to Handle Inconsistent Data

(ii) Handle Duplicate Records

- Identify duplicate rows:
 - Use functions like `drop_duplicates()` in tools like Python's pandas or SQL queries.
- Define rules to retain or merge duplicates:
 - Keep the first occurrence or aggregate values where necessary.

(iii) Resolve Conflicting Values

- Compare similar records and resolve conflicts using:
 - **Domain Knowledge:** Consult experts or reference sources to identify the correct values.
 - **Majority Rule:** Use the most frequent or likely value in conflicting records.

Techniques to Handle Inconsistent Data

(iv) Correct Typographical Errors

- Use text-cleaning techniques:
 - Spell-checking tools or libraries (e.g., fuzzywuzzy in Python).
 - String similarity measures (e.g., Levenshtein distance) to correct similar misspellings.

(v) Impute Missing Values

- Use imputation techniques to handle missing data:
 - **Basic Techniques (Mean/Median/Mode** for numerical, skewed, and categorical data respectively).
 - **Statistical Methods**
 - **Machine Learning based Methods.**

Techniques to Handle Inconsistent Data

(vi) Normalize Data

- Ensure consistent scaling for numerical data:
 - Apply normalization (e.g., min-max scaling) or standardization (z-scores).
- Transform data into a common structure:
 - Convert categorical data to one-hot encoding or label encoding.

(vii) Validate Data

- Define validation rules to check for inconsistencies:
 - **Ranges:** Ensure numerical values fall within expected ranges.
 - **Constraints:** Apply rules like unique IDs or non-negative values.

Techniques to Handle Inconsistent Data

(viii) Automate Cleaning Processes

- Use tools and scripts for automation:
 - Python libraries: pandas, NumPy, OpenRefine.
 - ETL (Extract, Transform, Load) tools: Talend, Alteryx.

(ix) Document the Cleaning Process

- Maintain detailed logs of the cleaning steps:
 - Record changes made to the dataset for transparency and reproducibility.

Tools for Handling Inconsistent Data

- **Programming Tools:**

- Python: pandas, NumPy, regex libraries.
- R: dplyr, tidyr.

- **Data Cleaning Software:**

- OpenRefine, Trifacta.

- **Excel/Google Sheets:**

- Use functions like TRIM, CLEAN, FIND, REPLACE.

- **Database Management Systems:**

- SQL for querying and cleaning data directly in databases.

Inconsistent Data

- There may also be inconsistencies due to **data integration**, where a given attribute can have different names in different databases.
- Data integration:
 - Combines data from multiple sources into a coherent store. These sources may include multiple databases, data cubes, or flat files.
 - There are a number of issues to consider during data integration:
 - i) Schema integration
 - ii) Detection and resolution of data value conflicts
 - iii) Redundancy

Schema integration

- **Schema integration** can be tricky. How can equivalent real-world entities from multiple data sources be matched up?
- This is referred to **Entity identification problem**. For example, how can the data analyst or the computer be sure that cust-id in one database and cust-number in another refer to the same entity?
- Databases and data warehouses typically have metadata (i.e., data about the data). Such metadata can be used to help avoid errors in schema integration. We need to integrate metadata from different sources

Detection and resolution of data value conflicts

- This is another issue in data integration.
- For example, for the same real world entity, attribute values from different sources may differ.
- Possible reasons: different representations, different scaling, or encoding.
- For instances:
 - i) a weight attribute may be stored in metric units in one system and British imperial units in another,
 - ii) The price of different hotels may involve not only different currencies but also different services (such as free breakfast) and taxes.
- Such semantic heterogeneity of data poses great challenges in data integration.

Redundancy

- Redundancy often occurs due to integration of multiple databases.
 - *Object identification*: The same attribute (e.g., cust.id and cust.#) or object (Bill Clinton and William Clinton) may have different names in different databases
 - *Derivable data*: An attribute may be redundant if it can be derived from another table, such as annual revenue.
- Some redundancies can be detected by *correlation analysis*
- Careful integration of the data from multiple sources may help reduce/avoid redundancies and inconsistencies and improve mining speed and quality

Correlation

- The word correlation denotes some form of association. Correlation between (i) foggy days and attacks of wheeziness, (ii) age and height of persons, etc.
- Correlation means association between two quantitative variables.
- Assume that the association is linear: i.e., one variable increases or decreases a fixed amount for a unit increase or decrease in the other.
- The other technique that is often used in these circumstances is regression, which involves estimating the best straight line to summarize the association.
- There are usually three types of correlation analysis: Pearson, Spearman, and Kendall. Spearman's Rank and Pearson's Coefficient are the two most widely used analytical formulae depending on the types of data researchers use.

Pearson Correlation coefficient

- The degree of association is measured by a correlation coefficient, denoted by r , where

$$r = \frac{\sum (x_i - \bar{x}) (y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

r = correlation coefficient

x_i = values of the x-variable in a sample

$\bar{x}$ = mean of the values of the x-variable

y_i = values of the y-variable in a sample

$\bar{y}$ = mean of the values of the y-variable

- It is called Pearson Product-Moment coefficient and is a measure of linear association. It determines the strength of the linear relationship between two variables.
- This is an dimensionless coefficient, meaning that there are no data-related boundaries to be considered when conducting analyses with this formula.

Pearson Correlation coefficient

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

- It could be hypothesized that number of hours the students study for a final examination is **positively** related to the score they will be received on the examination.

$$r = \frac{142}{\sqrt{28} \sqrt{1864}} = 0.62$$

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
2	58	-2	-2	4	4	4
4	32	0	-28	0	0	784
5	63	1	3	3	1	9
7	87	3	27	81	9	729
3	67	-1	7	-7	1	49
1	45	-3	-15	45	9	225
6	68	2	8	16	4	64
$\bar{x}=4$	$\bar{y}=60$			$\sum = 142$	$\sum = 28$	$\sum = 1864$

Rank correlation coefficient

- We have discussed the sample (Pearson's) correlation coefficient to measure the correlation between two sets of continuous random variables x and y .
- If instead we replace their numerical values by their rankings, then we obtain the *rank correlation coefficient* (this is due to Spearman and so is also called the *Spearman rank correlation coefficient*).

Spearman's Rank Correlation coefficient

- If the relationship between the data is not linear, Spearman's Rank Correlation Coefficient may be used to find the relationship between the two variables.
- Here, the assumption is that the data being used is ordinal, i.e., the numbers do not indicate quantity, but rather they signify a position like 1st, 2nd, 3rd, etc.
- It is defined as:
$$r_s = 1 - \frac{6 \sum D^2}{n(n^2 - 1)}$$
 re D is the

difference between two ranks.

Spearman's Rank Correlation coefficient

Students	Maths	Science
A	35	24
B	20	35
C	49	39
D	44	48
E	30	45

Step 1: Create a table for the given data.

Step 2: Rank both the data in descending order. The highest marks will get a rank of 1 and the lowest marks will get a rank of 5.

Step 3: Calculate the difference between the ranks (D) and the square value of D.

Students	Maths(X)	Rank(X)	Science(Y)	Rank(Y)	Difference(D)	D ²
A	35	3	24	5	2	4
B	20	5	35	4	1	1
C	49	1	39	3	2	4
D	44	2	48	1	1	1
E	30	4	45	2	2	4
						14

Step 4: Add all your D square values.

$$r = 1 - \frac{6 \sum D^2}{n(n^2 - 1)} = 1 - \frac{6 \times 14}{5 \times 24} = 0.3$$

The value is near 0, which means that there is a weak positive correlation between the two ranks.

Spearman's Rank Correlation coefficient

Object	X	Y
A	10	7
B	8	4
C	12	6
D	15	7
E	8	9
F	10	8

Object	X	Rank(X)	Y	Rank(Y)	Difference (D)	D ²
A	10	3.5	7	3.5	0	0
B	8	5.5	4	6	0.5	0.25
C	12	2	6	5	3	9
D	15	1	7	3.5	2.5	6.25
E	8	5.5	9	1	4.5	20.25
F	10	3.5	8	2	1.5	2.25
						38

$$r = 1 - \frac{6 \sum D^2}{n(n^2 - 1)} = 1 - \frac{6 \times 38}{6 \times 35} = -0.085714$$

So there is a Weak (or small) negative correlation between X and Y

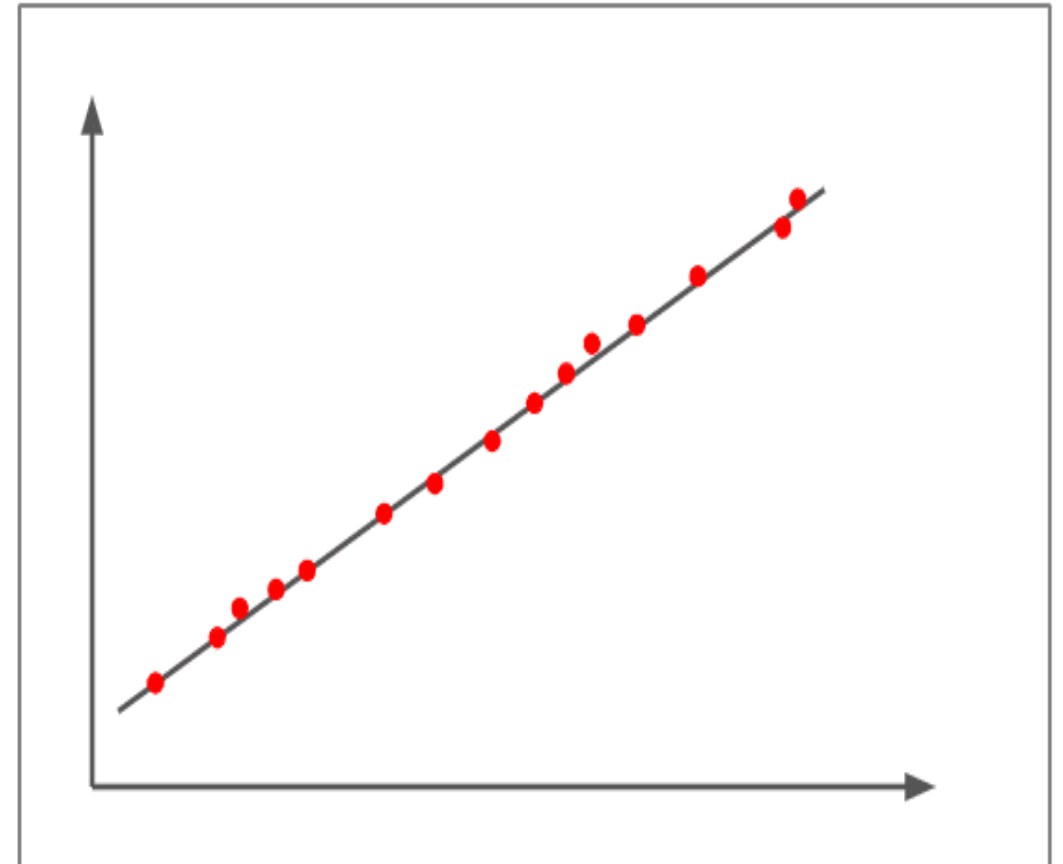
Correlation Coefficient

- In both the cases, the correlation coefficient varies from -1 through 0 to +1.
- Complete correlation between two variables is expressed by either + 1 or -1.
- When one variable increases as the other increases the correlation is positive
- when one decreases as the other increases it is negative.
- Complete absence of correlation is represented by 0.

Interpretation of Correlation Coefficient

1. Large positive correlation

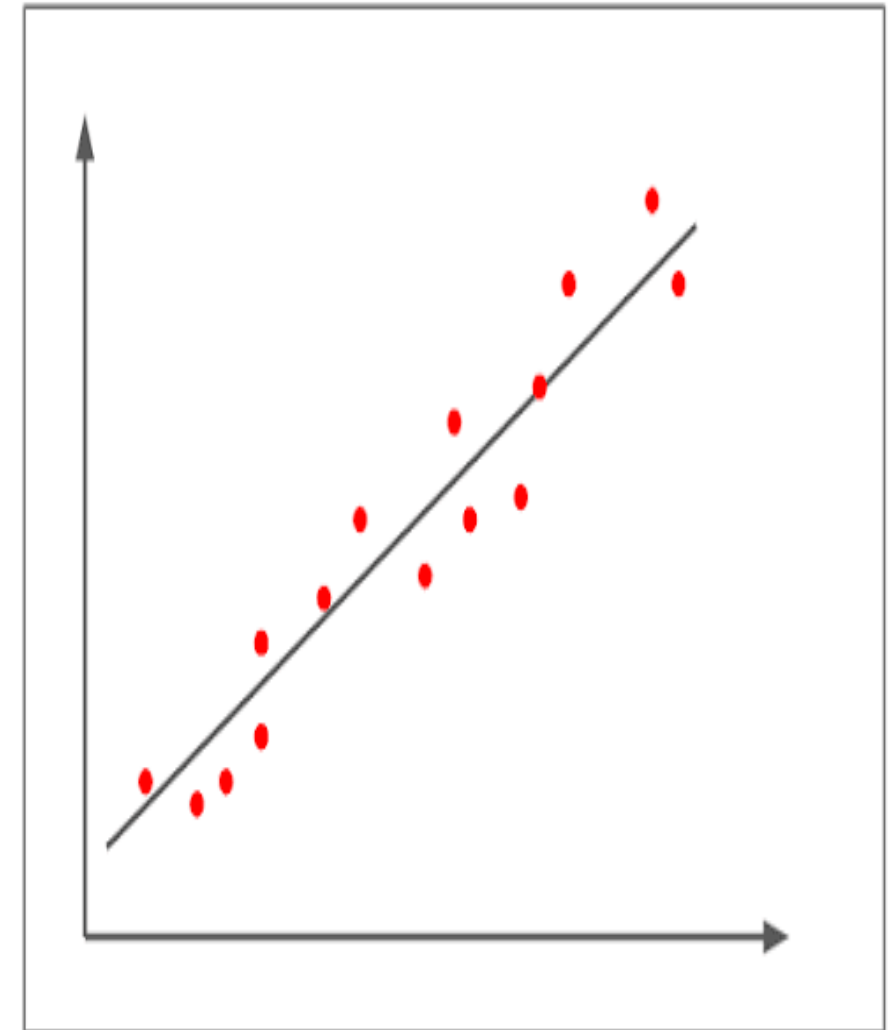
- The figure depicts a correlation of almost +1. (0.5 to +1)
- The scatter plots are nearly plotted in a straight line.
- The slope is positive, which means that if one variable increases, the other variable also increases, showing a positive linear line.
- This denotes that a change in one variable is directly proportional to the change in the other variable.
- Example – As children grow, so do their clothes and shoe sizes.



Interpretation of Correlation Coefficient

2. Medium positive correlation

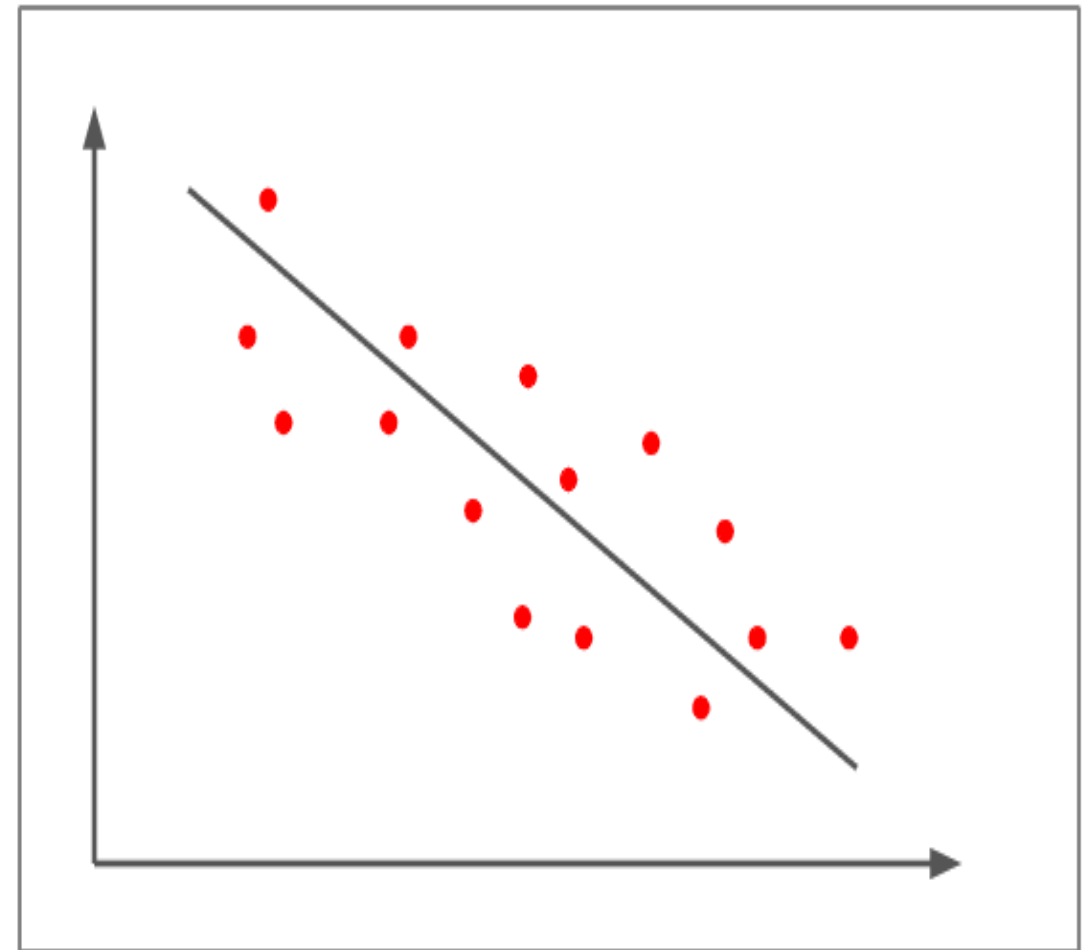
- The figure depicts a positive correlation.
- The correlation is above $+0.3$ but below $+0.5$.
- It shows a pretty strong linear uphill pattern.
- Example - As the number of automobiles increases, so makes the demand for the fuel variable increases.



Interpretation of Correlation Coefficient

3. Small negative correlation

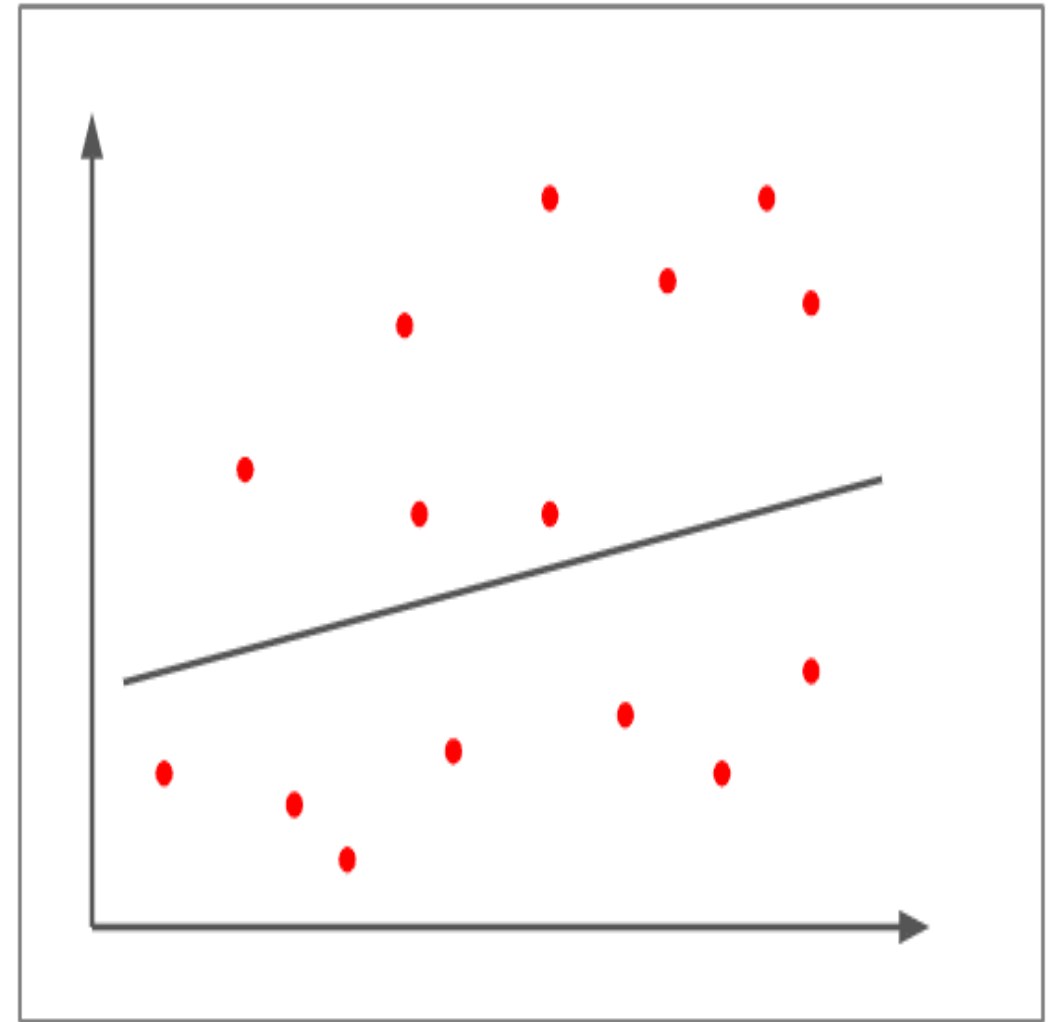
- In the figure, the scatter plots are not as close to the straight line compared to the earlier examples
- It shows a negative linear correlation of approximately -0.3
- The change in one variable is inversely proportional to the change in the other variable, as the slope is negative.
- Example – The more somebody eats, the less hungry they get.



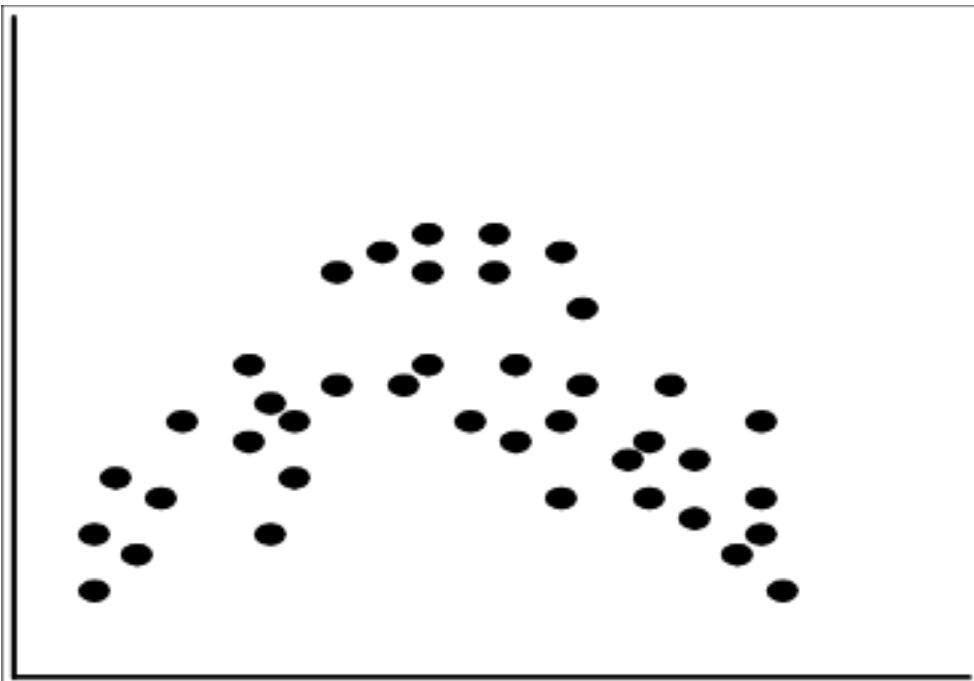
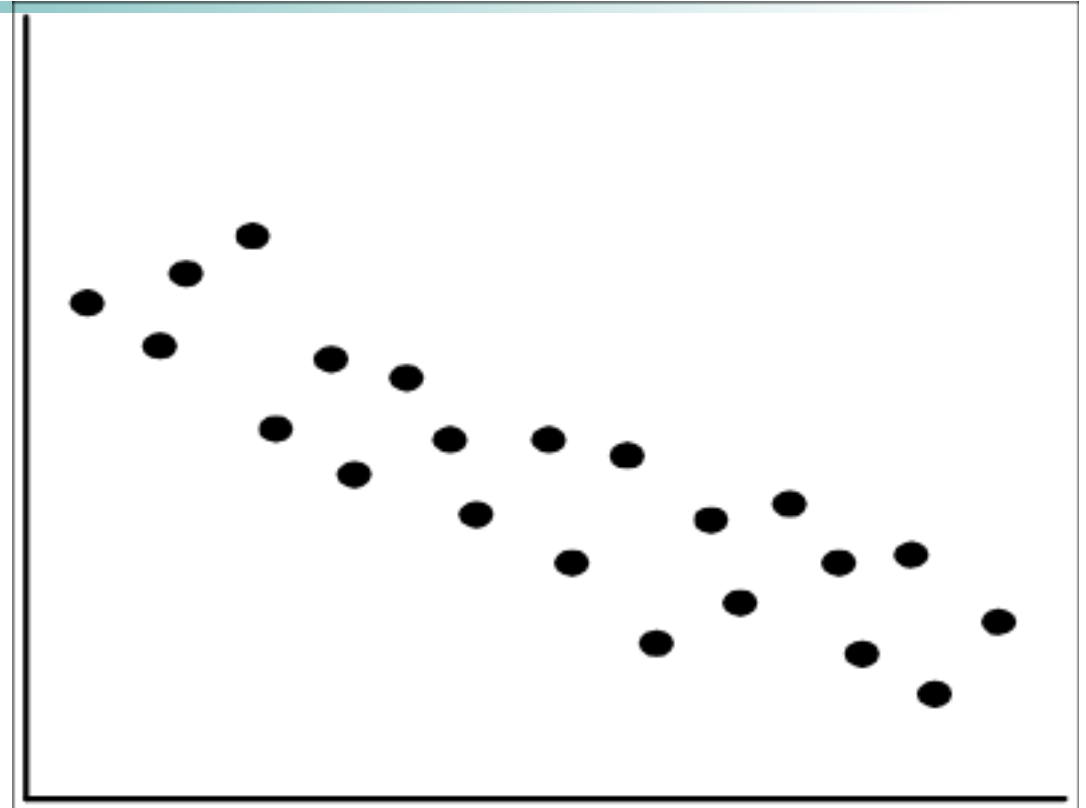
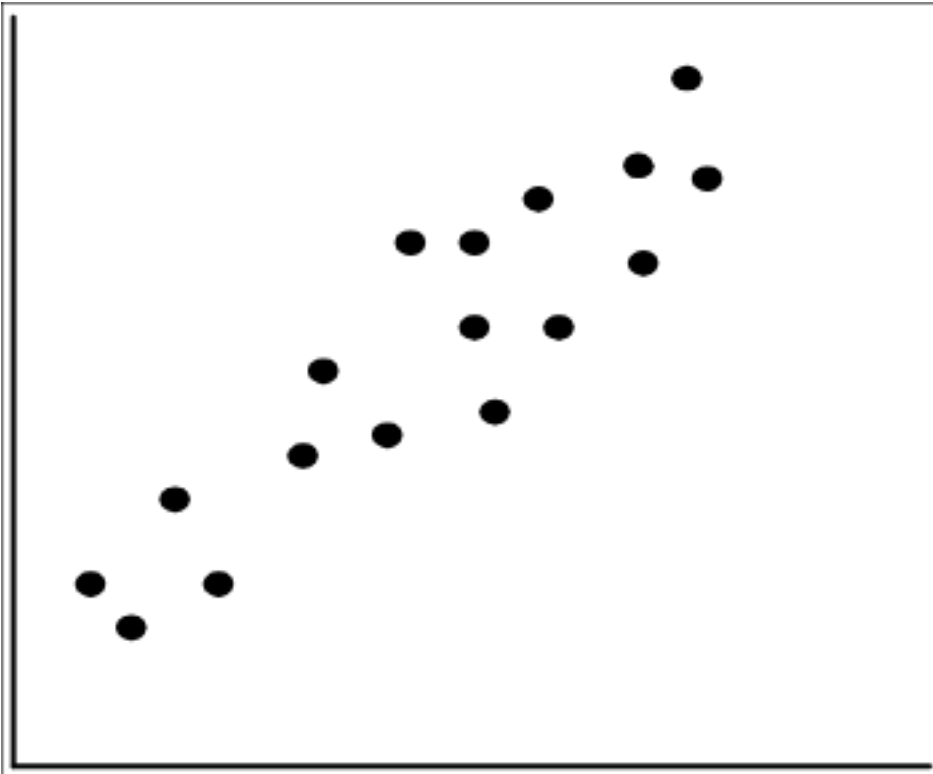
Interpretation of Correlation Coefficient

4. Weak / no correlation

- The scatterplots are far away from the line.
- It is tough to draw a line practically.
- The correlation is approximately $+0.15$
- It can't be judged that the change in one variable is directly proportional or inversely proportional to the other variable.
- Example – An increase in fuel prices leads to lesser people adopting pets.

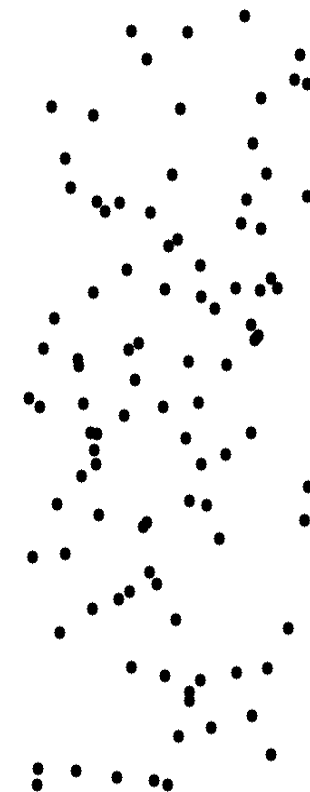
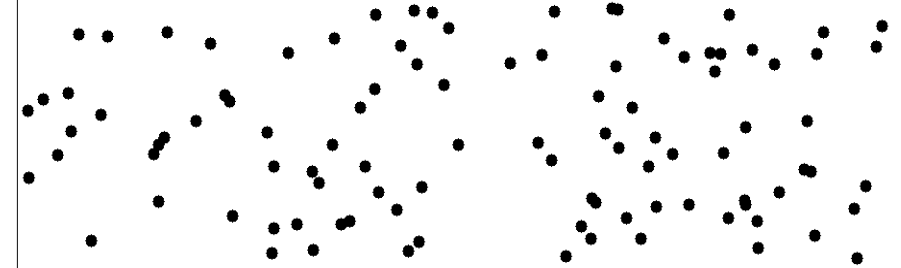
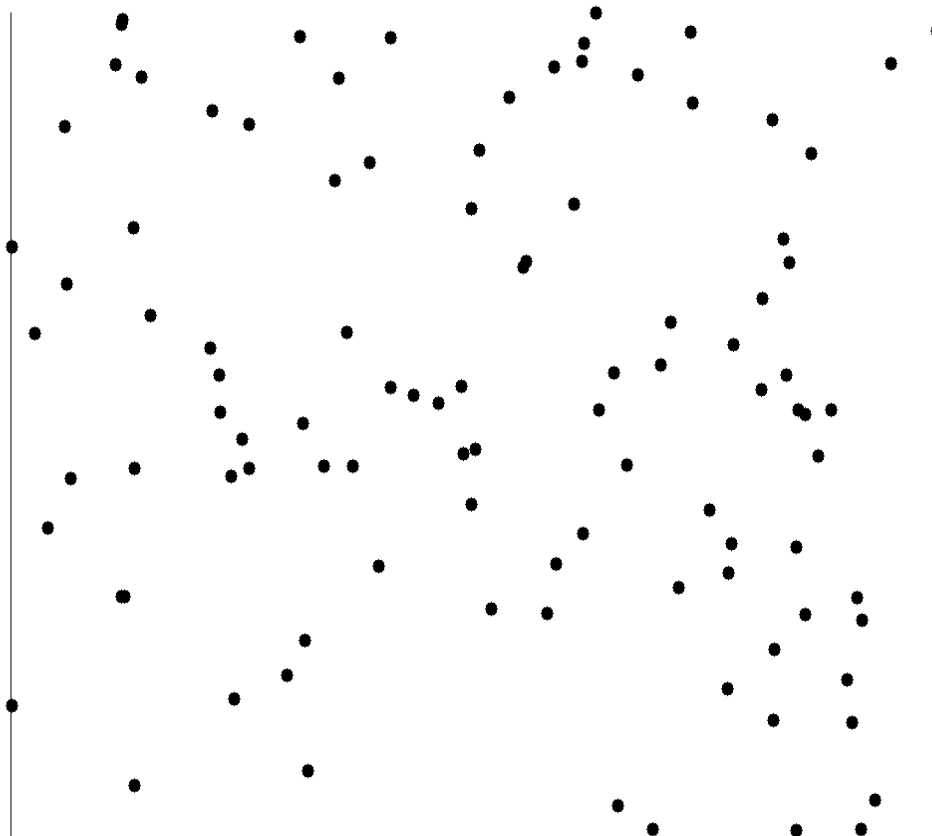


Positively and Negatively Correlated Data



- The left half fragment is positively correlated
- The right half is negative correlated

Uncorrelated Data



Interpretation of Correlation Coefficient

Strength of Association	Coefficient, r	
	Positive	Negative
Small	.1 to .3	-0.1 to -0.3
Medium	.3 to .5	-0.3 to -0.5
Large	.5 to 1.0	-0.5 to 1.0

- The closer the scatterplots lie next to the line, the stronger the relationship between the variables.
- The further they move from the line, the weaker the relationship gets.
- If the line is nearly parallel to the x-axis due to the scatterplots randomly placed on the graph, it's safe to assume that there is no correlation between the two variables.

Strength and Direction

- The terms 'strength' and 'direction' have statistical significance.
- **Strength:** Strength signifies the relationship correlation between two variables. It means how consistently one variable will change due to the change in the other.
- Values that are close to +1 or -1 indicate a strong relationship.
- These values are attained if the data points fall on or are very close to the line.
- The further the data points move away, the weaker the strength of the linear relationship.
- When there is no practical way to draw a straight line because the data points are scattered, the strength of the linear relationship is the weakest.

Direction

- **Direction:** The direction of the line indicates a positive linear or negative linear relationship between variables.
- If the line has an upward slope, the variables have a positive relationship.
 - This means an increase in the value of one variable will lead to an increase in the value of the other variable.
- A negative correlation depicts a downward slope.
 - This means an increase in the amount of one variable leads to a decrease in the value of another variable.

Outliers

- Outliers or anomalies must be accounted for in both correlation coefficients.
- Using a scatter graph is the easiest way of identifying any anomalies that may have occurred.
- Running the correlation analysis twice (with and without anomalies) is a great way to assess the strength of the influence of the anomalies on the analysis.
- If anomalies are present, Spearman's Rank coefficient may be used instead of Pearson's Coefficient, as this formula is extremely robust against anomalies due to the ranking system used.

When to Use

- These two methods are used according to whether there are parameters associated with the data gathered.
- Parametric: (*Pearson's Coefficient*) - Where the data must be handled in relation to the parameters of populations or probability distributions. Typically used with quantitative data already set out within said parameters.
- Nonparametric: (*Spearman's Rank*) - Where no assumptions can be made about the probability distribution. Typically used with qualitative data, but can be used with quantitative data if Spearman's Rank proves inadequate.
- In cases when both are applicable, statisticians recommend using the parametric methods such as Pearson's Coefficient, because they tend to be more precise. But that doesn't mean discount the non-parametric methods if there isn't enough data or a more specified accurate result is needed.

Data Transformation

- Smoothing: remove noise from data
- Aggregation: summarization, data cube construction
- Generalization: concept hierarchy climbing
- Normalization: scaled to fall within a small, specified range
 - min-max normalization
 - z-score normalization
 - normalization by decimal scaling
- Attribute/feature construction
 - New attributes constructed from the given ones

Data Transformation: Normalization

- Min-max normalization: to $[\text{new_min}_A, \text{new_max}_A]$

$$v' = \frac{v - \min A}{\max A - \min A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

- Ex. Let income range \$12,000 to \$98,000 normalized to $[0.0, 1.0]$. Then \$73,600 is mapped to $\frac{73,600 - 12,000}{98,000 - 12,000} (1.0 - 0) + 0 = 0.716$

- **When to use:**

- When features have a known and consistent range (e.g., pixel values in images, measurement units).
- When using algorithms like K-Nearest Neighbors (KNN) and Neural Networks that are sensitive to the scale of features.
- When you need to preserve the relationship between the data points in the range $[0, 1]$.

Data Transformation: Normalization

- Z-score normalization (μ : mean, σ : standard deviation):

$$v' = \frac{v - \mu_A}{\sigma_A}$$

- Ex. Let $\mu = 54,000$, $\sigma = 16,000$. Then $v = 73,600$ is normalized to

$$\frac{73,600 - 54,000}{16,000} = 1.225$$

- **When to use:**

- When the data follows a Gaussian (normal) distribution or when you assume that it should be centered around 0.
- For algorithms like Logistic Regression, Linear Regression, and Support Vector Machines (SVM) that assume normally distributed data.
- When the range of the data varies significantly across features.

Normalization by decimal scaling

$$v' = \frac{v}{10^j} \quad \text{Where } j \text{ is the smallest integer such that } \text{Max}(|v'|) < 1$$

- Suppose we have the following data: [10,200,300,400]
- The maximum absolute value is 400, so we choose $k=3$ and dividing the maximum value (400) by 1000 will bring it below 1.
- After applying decimal scaling normalization, the transformed data would be: [0.01,0.2,0.3,0.4]
- **When to use:**
 - **Simple to apply:** It's straightforward and doesn't require knowledge of the distribution of the data.
 - **Effective for bounded data:** It works well when the data needs to be scaled within a certain range, especially when the maximum value is known.

Disadvantages of Decimal Scaling Normalization

- **Sensitive to the maximum value:** The method depends on the largest absolute value in the dataset, which may not always be ideal if the dataset contains outliers.
- **May distort the data:** If there are extreme outliers, the scaling may cause loss of information or lead to disproportionate scaling of values.
- This method is less commonly used in practice compared to other normalization techniques like Min-Max Scaling or Z-score normalization but can be effective for certain applications where scaling to a particular range is needed.

Robust Scaling

- **Description:** This technique scales the data using the median and interquartile range (IQR), making it robust to outliers. The formula is:

$$X' = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

- **When to use:**
 - When the dataset contains many outliers, as it is less sensitive to them compared to Min-Max Scaling and Z-score Normalization.
 - When the data does not follow a Gaussian distribution and the range of data can vary widely.

Max Abs Scaling

- **Description:** This method scales each feature by its maximum absolute value, so that the range of each feature is between $[-1, 1]$. The formula is:

$$X' = \frac{X}{\max(|X|)}$$

- **When to use:**
 - When your data is already centered around 0 but with varying magnitudes, or when you want to preserve the sign of the values.
 - In algorithms like SVM and Neural Networks, especially when sparse data is involved.

Log Transformation

- **Description:** This technique applies the logarithmic function to the data, often used for reducing the impact of large values. The formula is:

$$X' = \log(X)$$

- **When to use:**
 - When data has exponential growth or a skewed distribution.
 - For financial data or counts, where values can vary over several orders of magnitude.
 - When you need to reduce the effect of extreme values or outliers in datasets like population or income data.

Unit Vector Normalization (L2 Normalization)

- **Description:** This technique scales each data point by dividing it by its Euclidean norm (the magnitude of the vector). The formula is:

$$X' = \frac{X}{\|X\|}$$

- **When to use:**
 - When the data needs to be expressed as a unit vector (magnitude 1), often used in text mining and document similarity tasks (e.g., TF-IDF features).
 - In clustering and nearest neighbor algorithms where the direction of the data point is more important than its magnitude.

Summary of When to Use Each Technique

- **Min-Max Scaling:** When the features have a known range, and you need to preserve relationships between values (e.g., image data, neural networks).
- **Z-score Normalization:** When the data is normally distributed, or the model assumes normally distributed data (e.g., linear models, SVM).
- **Decimal Scaling:** It works well when the data needs to be scaled within a certain range, especially when the maximum value is known.
- **Robust Scaling:** When the data contains outliers or is not normally distributed (e.g., financial data, certain regression models).

Summary of When to Use Each Technique

- **Max Abs Scaling:** When the data is already centered around 0 and needs scaling within the range $[-1, 1]$ (e.g., sparse data).
- **Log Transformation:** When the data has exponential growth or is highly skewed (e.g., income, population data).
- **Unit Vector Normalization:** When you want to scale data to unit vectors, especially in machine learning tasks involving text or document similarity (e.g., clustering, nearest neighbor search).
- By choosing the appropriate normalization technique based on the characteristics of the dataset and the algorithm, you can improve model performance and accuracy.

Need of Normalization

- Normalization is generally required when we are dealing with attributes on a different scale, otherwise, it may lead to a dilution in effectiveness of an equally important attribute (on lower scale) because of other attribute having values on larger scale.
- In simple words, when multiple attributes are there but attributes have values on different scales, this may lead to poor data models while performing data mining operations.
- So they are normalized to bring all the attributes on the same scale.

Need of Normalization

person_name	Salary	Year_of_experience	Expected Position Level
Aman	100000	10	2
Abhinav	78000	7	4
Ashutosh	32000	5	8
Dishi	55000	6	7
Abhishek	92000	8	3
Avantika	120000	15	1
Ayushi	65750	7	5

The attributes salary and year_of_experience are on different scale and hence attribute salary can take high priority over attribute year_of_experience in the model.

ADVANTAGES OF NORMALIZATION

- **Improved performance of machine learning algorithms:** Normalization can help to improve the performance of machine learning algorithms by scaling the input features to a common scale.
- **Better handling of outliers:** Normalization can help to reduce the impact of outliers by scaling the data to a common scale, which can make the outliers less influential.
- **Improved interpretability of results:** Normalization can make it easier to interpret the results of a machine learning model, as the inputs will be on a common scale.
- **Better generalization:** Normalization can help to improve the generalization of a model, by reducing the impact of outliers and by making the model less sensitive to the scale of the inputs.

DISADVANTAGES OF NORMALIZATION

- **Loss of information:** Normalization can result in a loss of information if the original scale of the input features is important.
- **Impact on outliers:** Normalization can make it harder to detect outliers as they will be scaled along with the rest of the data.
- **Impact on interpretability:** Normalization can make it harder to interpret the results of a machine learning model, as the inputs will be on a common scale, which may not align with the original scale of the data.
- **Additional computational costs:** Normalization can add additional computational costs to the data mining process, as it requires additional processing time to scale the data.

pros and cons of data normalization

- In conclusion, data normalization can have both advantages and disadvantages.
- It can improve the performance of machine learning algorithms and make it easier to interpret the results.
- However, it can also result in a loss of information and make it harder to detect outliers.
- It's important to weigh the pros and cons of data normalization and carefully assess the risks and benefits before implementing it.

References

- D. P. Ballou and G. K. Tayi. Enhancing data quality in data warehouse environments. *Communications of ACM*, 42:73-78, 1999
- T. Dasu and T. Johnson. *Exploratory Data Mining and Data Cleaning*. John Wiley & Sons, 2003
- T. Dasu, T. Johnson, S. Muthukrishnan, V. Shkapenyuk. [Mining Database Structure; Or, How to Build a Data Quality Browser](#). SIGMOD'02.
- H.V. Jagadish et al., Special Issue on Data Reduction Techniques. *Bulletin of the Technical Committee on Data Engineering*, 20(4), December 1997
- D. Pyle. *Data Preparation for Data Mining*. Morgan Kaufmann, 1999
- E. Rahm and H. H. Do. Data Cleaning: Problems and Current Approaches. *IEEE Bulletin of the Technical Committee on Data Engineering*. Vol.23, No.4
- V. Raman and J. Hellerstein. *Potters Wheel: An Interactive Framework for Data Cleaning and Transformation*, VLDB'2001
- T. Redman. *Data Quality: Management and Technology*. Bantam Books, 1992
- Y. Wand and R. Wang. Anchoring data quality dimensions ontological foundations. *Communications of ACM*, 39:86-95, 1996
- R. Wang, V. Storey, and C. Firth. A framework for analysis of data quality research. *IEEE Trans. Knowledge and Data Engineering*, 7:623-640, 1995

Thank You!